MedlyDose

Resilience Report: a medicine reminder that fixes itself

Chhavi Bansal | BCA3 | Self-healing software hackathon

1. The case

What MedlyDose is. A daily medicine and reminder app. A person adds each medicine with its times; at each due time the app alerts them on screen and by text message, they tick the dose off as Taken, and a caregiver view shows it was done. A calendar shows what was taken, missed and still to come.

Who gets hurt when it fails. People on time-critical medicines (diabetes, blood pressure, the heart), older people who rely on a reminder, and the relatives who trust the caregiver view. A missed dose can undo a day of treatment and a doubled dose can be dangerous, yet these apps fail at the worst moments: the internet drops, the text provider has an outage, the program crashes overnight.

The challenge we set ourselves. Make the app notice its own failures, work out what went wrong, and recover within 30 seconds with nobody restarting anything. Then break it on purpose, on camera, and record the times. This report is the evidence.

What the working app does

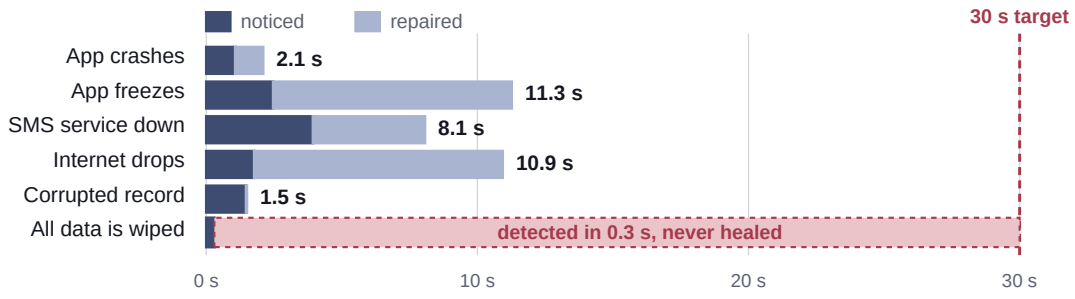
- Several times a day per medicine, repeating every day, with a Remind me bell before the due time and a Taken button once it is due.
- A text message and an on-screen alert at the due time. The on-screen alert comes from the page itself, so it works even when the text service is down.
- A history of everything taken, a calendar of every day, and a caregiver view that receives each taken dose.
- A live Health page: each part of the system shown green, amber or red, an incident log, and a count of recoveries.
- A Chaos panel and a `chaos.py` script that break each part on demand, plus a switch that turns self-healing off so the same failure can be shown with and without protection.

Results at a glance

6	5 of 5	11.3 s	0
failures injected	healable failures healed	slowest average recovery	people needed to help

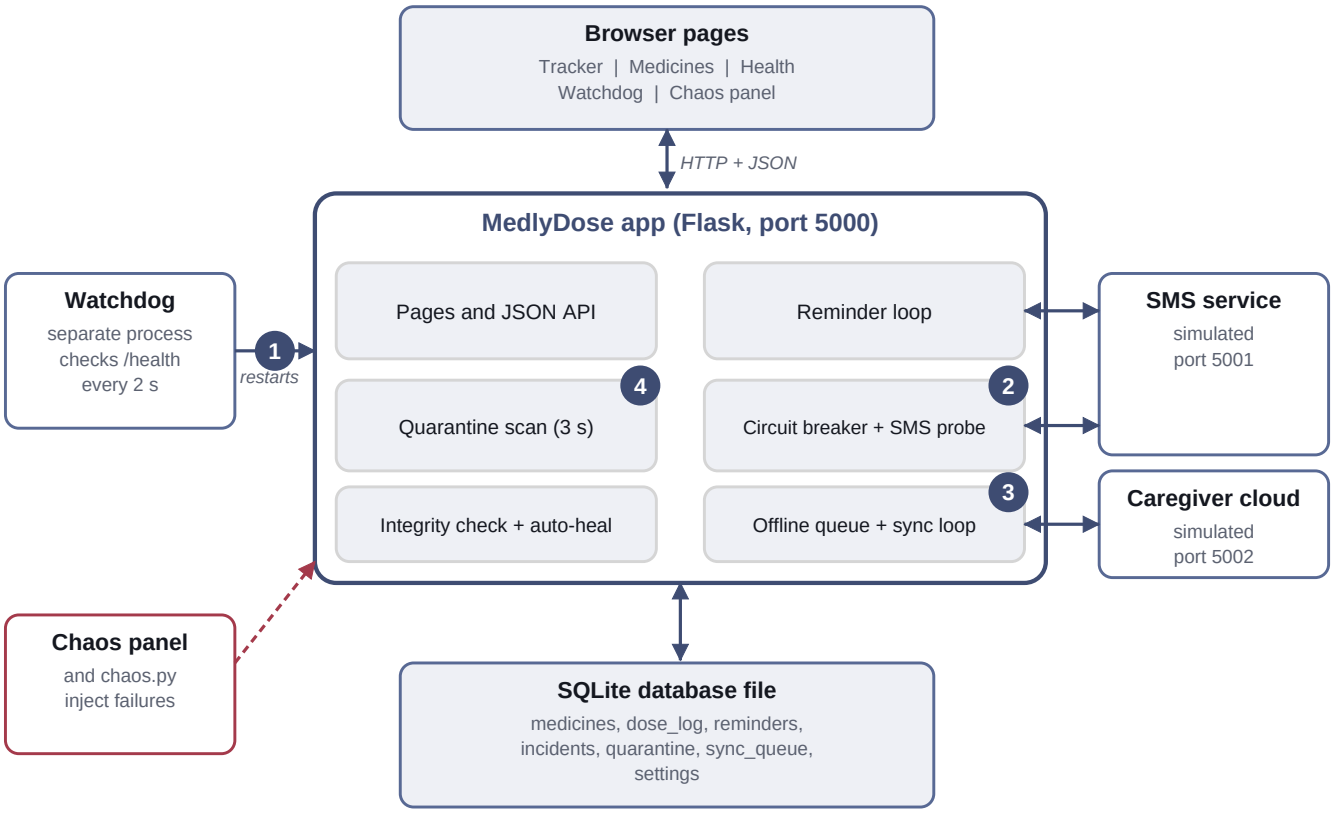
Four self-healing mechanisms (watchdog, circuit breaker, offline queue, data quarantine) noticed and repaired every healable failure on their own: 15 of 15 runs finished inside the 30 second target. **The honest limit:** wiped data cannot be restored, because there is no backup.

How fast each failure was fixed (average of 3 runs, seconds)



2. Architecture

MedlyDose is a small Flask web app that keeps everything in one SQLite file. Two helper services stand beside it (a text message service and a caregiver cloud, both simulated for the hackathon), and a separate watchdog program guards it. The numbered badges mark where each of the four healing mechanisms sits.



The chaos panel and the watchdog are outside the app on purpose: the thing that repairs the app must not live inside the app.

Why these four mechanisms

#	Mechanism	Failure it covers	Why we chose it
1	**Watchdog: health checks and automatic restart**	The app crashes or freezes	It is the one failure that stops everything, and a program cannot restart itself once it is dead. A separate process checks `/health` every 2 s and restarts the app after the process exits or after 3 failed checks in a row.
2	**Circuit breaker**	The text message service stops answering	Calling a dead service again and again wastes time and hides the real problem. After 3 failures the app stops calling it, tests it again after a short rest, and resumes once it answers. Waiting reminders are sent afterwards, not lost.
3	**Offline-first queue with later sync**	The internet to the caregiver drops	A dose taken must never be lost because the connection was down. It is saved on the device first, then sent to the caregiver oldest-first when the connection returns. Sending the same dose twice is harmless.
4	**Self-repairing data (quarantine)**	A corrupted or invalid record	One bad record must not break the whole medicine list. Every record is validated every 3 s. A bad one is moved to a quarantine table and the rest keeps working.

Supporting pieces. Every incident is written to an incident log as Detected, Diagnosed, Action taken, Recovered, with timestamps. An integrity check watches for a sudden loss of medicines. When self-healing is on, a helper service that stays down is reconnected automatically after about 6 to 8 seconds. A settings flag turns all of this off for the with-and-without comparison.

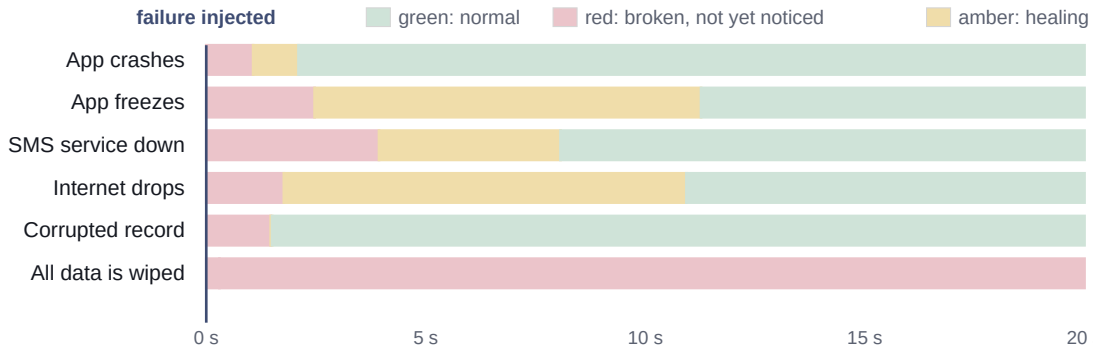
3. Failures, detection and recovery

Each failure was injected with the chaos script. Times are in seconds from the moment of injection and are the average of 3 runs, with the slowest run in brackets. **Detected** is how long the system took to notice; **Recovered** is how long until it was back to normal with no human action.

Failure	How it is detected	How it heals	Detected (s)	Recovered (s)
App crashes	The watchdog sees the app process has exited.	Starts a new app process and waits until it answers.	1.1 (2.1)	2.1 (3.2)
App freezes	The watchdog's health check gets no answer, 3 times in a row (2 s apart).	Stops the frozen process and starts a fresh one.	2.5 (2.6)	11.3 (11.4)
SMS service goes down	The SMS probe fails 3 times, so the circuit breaker opens.	Stops calling the service, reconnects it automatically, tests it again, sends the waiting text.	3.9 (4.0)	8.1 (8.2)
Internet to the caregiver drops	The sync loop pings the cloud every 3 s and gets no answer.	Saves doses on the device, reconnects the link automatically, syncs oldest-first.	1.8 (2.0)	10.9 (11.1)
A corrupted record enters the database	The 3-second quarantine scan validates every record.	Moves the bad record to quarantine. The rest keeps working.	1.5 (1.6)	1.5 (1.7)
All data is wiped	The integrity check sees healthy medicines suddenly missing.	Cannot heal. Raises the alarm and marks the incident Not healed.	0.3 (0.3)	**Never** not healed

Averages over 3 runs each (the data-wipe test ran once). Measured on one test computer; times on another machine will differ a little.

What the Health page showed, second by second



Built from the measured averages. Every healable failure turns amber the moment it is noticed and green again with nobody touching anything. The data wipe stays red.

What an incident receipt looks like

1 Detected — the problem is seen

2 Diagnosed — what went wrong is worked out

3 Action taken — the fix is applied

4 Recovered — back to normal, time recorded

Every failure leaves the same four-part record, written automatically. This is a real one from the test run:

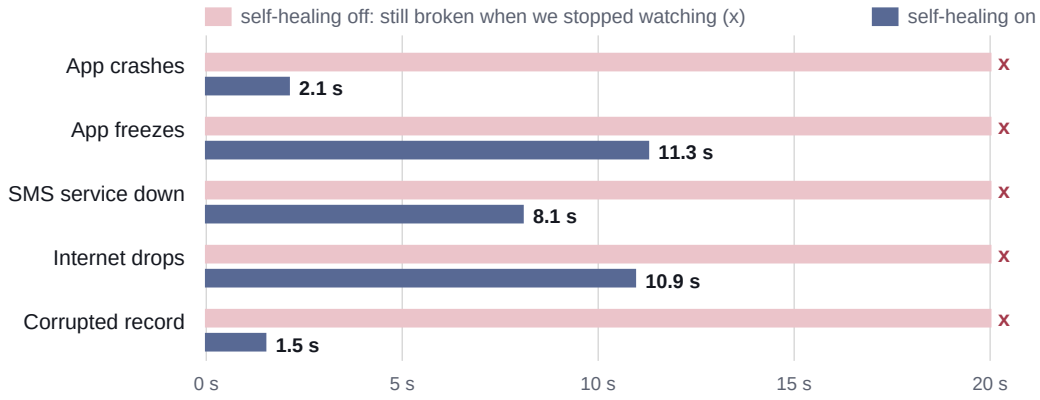
Detected	Diagnosed	Action taken	Recovered
09:16:33.623	sms_service failed 3 times in a row	Circuit opened: stopped calling it, testing again every 10s	09:16:37.587 after 4.0 s

The Health page shows the latest incidents live. `python show_incidents.py` prints the full log.

4. Before and after

The Chaos panel has a Self-healing switch. Switched off, every mechanism stops: the watchdog does not restart, the circuit breaker does not protect, nothing queues, nothing is quarantined, and failed services are not reconnected. We ran the same failures both ways. With self-healing off each failure was watched for 20 seconds.

Failure	Without self-healing	With self-healing
App crashes	App still down after 20 s. It stays down until a person restarts it.	Back to normal in 2.1 s on average (slowest 3.2 s). No one touched anything.
App freezes	App still frozen after 20 s. Nothing noticed.	Back to normal in 11.3 s on average (slowest 11.4 s). No one touched anything.
SMS service goes down	The text failed and was never sent, even after the service came back.	Back to normal in 8.1 s on average (slowest 8.2 s). No one touched anything.
Internet to the caregiver drops	The dose never reached the caregiver. It was dropped.	Back to normal in 10.9 s on average (slowest 11.1 s). No one touched anything.
A corrupted record enters the database	The bad record stays in the database, mixed with real data.	Back to normal in 1.5 s on average (slowest 1.7 s). No one touched anything.
All data is wiped	Same result either way: the data is gone.	Detected in 0.3 s. Cannot be healed.



Reading the numbers

- Without self-healing, none of the five healable failures was repaired, and the two outages (text service and internet) caused real loss: a reminder text and a caregiver record that are gone for good.
- With self-healing on, 15 of 15 healable runs recovered inside the 30-second target, with no human action.
- The slowest recoveries (the text service and the internet link) are slower on purpose: the app waits a few seconds before reconnecting, and the circuit breaker rests between tests, so it does not hammer a service that is still down.

One thing to be clear about

With self-healing off and the text service down, the person still sees the on-screen reminder at the due time. That alert is made by the page in the browser and does not use the text service. What fails is the *extra* text message to the phone, which is lost. The reminder text is a second channel for people who are away from the screen, and that second channel is what the healing protects.

How to repeat the measurements

Start three terminals (`python watchdog.py`, `python mock_sms\server.py`, `python mock_cloud\server.py`), then run `python chaos.py` in a fourth. It runs every failure 3 times with self-healing on and once off, prints the table, and saves it. `python chaos.py inject crash` breaks one thing for a live demo.

5. Known limits and what we would build next

Good engineers earn trust by knowing the limits of what they built. These are ours.

The honest scoreboard

App crashes	App freezes	SMS down	Internet drops	Corrupt record	Data wiped
healed in 2.1 s	healed in 11.3 s	healed in 8.1 s	healed in 10.9 s	healed in 1.5 s	seen in 0.3 s *cannot be healed*

What it cannot heal

Limit	Why	What would fix it
Wiped data cannot be restored	The app notices within a second and raises the alarm, but the medicines and history are stored in one file with no backup. Nothing can recover data that no longer exists anywhere. We keep this as our honest, demonstrable limit.	Automatic backups to a second place, with a restore step. (Building that would make this failure healable, so we would then pick another honest limit.)
The watchdog itself is not guarded	Nothing restarts the watchdog. If it dies, the app keeps running but would not be restarted after a crash. The Health page turns red to say so.	Run the watchdog as an operating system service that the system restarts, or a second watchdog watching the first.
The helper services are simulated	The text and caregiver services are local stand-ins. A real provider adds network delays, rate limits and billing that we have not tested.	Connect a real SMS provider, and add retries that wait longer after each failure.
No protection against a sudden surge of traffic	We did not build or test protection for results-day style spikes. One computer and one database file would be slow under heavy load.	Rate limiting, a queue in front of the app, and a database built for many users.
No login	Anyone who can open the page on this computer can see and change the medicines.	Per-person accounts, with each person's data kept separate.
Measured on one machine	Times come from one test computer and 3 runs per failure. Another computer will give slightly different numbers.	Repeat the runs on several computers and report the spread.

What we would build next

- Missed-dose detection: if a dose is not taken, alert the caregiver automatically.
- Backups and restore, then a new honest limit to test against.
- A real SMS provider, with retries that back off.
- Automatic rollback: if a new version fails its health check, return to the last good one.
- A phone app, so the reminder reaches people who are away from the computer.

About this project

MedlyDose was built for the self-healing software hackathon by Chhavi Bansal, using Python, Flask and SQLite. Claude (Anthropic) was used as a coding assistant; the AI Use Declaration submitted with this entry lists what it was asked to do and what was written, changed and checked by hand.